



Development Guidelines

Guia de Desenvolvimento e Submissão

Índice

1. Objetivo do Projeto	2
2. Regras de Avaliação do Projeto	3
2.1. Componente de avaliação	3
2.2. Requisitos obrigatórios para apresentação e defesa	3
2.3. Entrega do Projeto	4
2.4. Situações que conduzem à classificação de 0 valores	4
2.5. Penalizações aplicáveis	4
2.6 Controlo de Versões (Git)	5
3. Regras de Desenvolvimento	7
3.1 Regras de Desenvolvimento do Frontend	7
3.2 Regras de Desenvolvimento da Base de Dados	11
3.3 Regras de Desenvolvimento do Backend	12
3.4 Estrutura de Diretórios e Organização do Projeto	14
3.5 Aspetos valorizados no desenvolvimento	15
3.6. Regras de Submissão	15



Este documento define a estrutura geral do projeto, bem como as regras de desenvolvimento e avaliação. Algumas secções serão detalhadas progressivamente ao longo do semestre, de acordo com os conteúdos lecionados na unidade curricular (Frontend, Bases de Dados e Backend).

A leitura integral deste documento é obrigatória.

A submissão do projeto pressupõe o conhecimento e a aceitação integral das regras nele definidas, não podendo o seu desconhecimento ser invocado como justificação para o incumprimento das mesmas.

1. Objetivo do Projeto

Cada estudante deverá desenvolver **individualmente** uma aplicação web que simule um **sistema de gestão do inventário hospitalar de equipamentos médicos**.

O sistema deverá permitir registar e gerir informação relevante sobre equipamentos, fornecedores e outros elementos associados ao funcionamento de um inventário hospitalar.

O objetivo não consiste apenas em criar uma lista de equipamentos, mas sim em desenvolver uma **estrutura de aplicação web** organizada, que possa servir de base à futura implementação de um sistema de gestão de manutenção de equipamentos médicos.

O projeto será **desenvolvido progressivamente ao longo do semestre**, acompanhando os conteúdos lecionados nas aulas:

- **Frontend (HTML, CSS, Bootstrap e JavaScript)**
- **Bases de Dados (MySQL)**
- **Backend (PHP)**

Algumas regras de desenvolvimento serão detalhadas à medida que cada módulo for introduzido.

2. Regras de Avaliação do Projeto

2.1. Componente de avaliação

O projeto corresponde à **componente M1-NR — Projeto**, com peso de **60% na classificação final da unidade curricular**.

A classificação final da unidade curricular é calculada de acordo com a fórmula definida na Ficha da Unidade Curricular:

$$CF = M1 \times 0.6 + EG \times 0.4$$

onde:

- M1 — Classificação da componente **Projeto (M1-NR)**
- EG — Classificação do **Exame Global**

Nota mínima exigida em cada componente: **8,5 valores**.

2.2. Requisitos obrigatórios para apresentação e defesa

Para que o estudante possa realizar a **apresentação e defesa do projeto**, deverão estar previamente cumpridos os seguintes requisitos:

- Submissão do **código do projeto na plataforma Moodle**, dentro do prazo definido.
- Entrega do **relatório técnico individual**.
- Entrega da **declaração de utilização de Inteligência Artificial**, devidamente preenchida e assinada.

A apresentação e defesa do trabalho constituem requisito obrigatório de avaliação, sendo condição necessária para a atribuição de classificação. A não comparência à apresentação e defesa do projeto, sem motivo devidamente justificado nos termos regulamentares da instituição, implica a **atribuição da classificação de 0 (zero valores)** na componente M1-NR — Projeto, mesmo que o trabalho tenha sido previamente submetido, **não sendo possível proceder à sua avaliação posterior**.

Durante a apresentação oral, cada estudante deverá demonstrar **domínio integral do trabalho** submetido, sendo capaz de:

- Explicar a estrutura da aplicação desenvolvida
- Justificar as decisões técnicas tomadas
- Demonstrar o funcionamento das principais funcionalidades
- Responder às questões colocadas sobre o código e o desenvolvimento do projeto



2.3. Entrega do Projeto

A entrega do projeto deverá ocorrer **até à data e hora definidas**.

- A submissão efetuada **até 30 minutos após o termo do prazo** implicará uma **penalização de 25% na classificação da componente M1-NR — Projeto**.
- Ultrapassado esse período, **a entrega não será aceite** e será atribuída **classificação de 0 (zero valores) na componente M1-NR — Projeto**.

2.4. Situações que conduzem à classificação de 0 valores

Será atribuída a classificação de **0 (zero valores)** na componente M1-NR — Projeto nas seguintes situações:

- **Não entrega da declaração de utilização de Inteligência Artificial**, devidamente preenchida e assinada.
- **Deteção de plágio**, designadamente a submissão de código total ou parcialmente produzido por terceiros ou copiado de outras fontes sem referência adequada.
- **Não entrega do trabalho dentro do prazo definido**.
- **Não comparência à apresentação e defesa do projeto**, sem motivo devidamente justificado nos termos regulamentares da instituição.
- **Incapacidade de explicar ou demonstrar domínio do trabalho** submetido durante a apresentação e defesa oral.

2.5. Penalizações aplicáveis

Serão aplicadas penalizações na **classificação da componente M1-NR — Projeto** nas seguintes situações:

- **Não utilização de controlo de versões com Git**, conforme definido nas regras do projeto: **penalização de 25% na classificação da componente M1-NR — Projeto**.

- **Número de commits inferior ao mínimo definido nas regras de desenvolvimento:** penalização de até 25%, a definir pelo docente em função da gravidade da situação, **na classificação da componente M1-NR — Projeto.**
- **Commits realizados apenas no final do semestre, sem evolução ao longo do tempo:** o histórico de commits **não será considerado evidência de desenvolvimento progressivo do projeto**, podendo resultar numa **penalização de até 25%**, a definir pelo docente em função da gravidade da situação, **na classificação da componente M1-NR — Projeto.**

O histórico de commits poderá ser analisado como evidência do processo de desenvolvimento do trabalho ao longo do semestre.

O docente poderá solicitar ao estudante que explique qualquer parte do código apresentado. A incapacidade de o fazer poderá ser interpretada como **ausência de domínio do trabalho submetido.**

2.6 Controlo de Versões (Git)

Durante o desenvolvimento do projeto, o estudante deverá utilizar **Git** para realizar o controlo de versões do código.

O repositório **deverá** refletir a **evolução real do desenvolvimento do projeto ao longo do semestre.**

- **Configuração obrigatória do Git**

A configuração do Git deve utilizar o nome e o email institucional do estudante, garantindo a identificação individual da autoria dos commits.

Exemplo de configuração:

```
git config --global user.name "Nome do Estudante"  
git config --global user.email "email@isep.ipp.pt"
```

Cada estudante deverá utilizar **uma configuração pessoal e única.** Commits realizados com identidades genéricas, partilhadas ou não identificáveis poderão **não ser considerados válidos para efeitos de avaliação.**

Regras

- Devem ser realizados **commits regulares ao longo do desenvolvimento do projeto**, refletindo a evolução do trabalho.
- Cada commit deverá representar **uma alteração significativa ou funcionalidade implementada**.
- Commits com mensagens como *"teste"*, *"update"* ou semelhantes **não serão considerados válidos**.
- O histórico de commits deverá evidenciar um **desenvolvimento progressivo ao longo do tempo**, não sendo considerados válidos históricos com submissões concentradas num único dia.

Registo do histórico de commits

O ficheiro (*commits.txt*) de histórico de commits deverá incluir informação **sobre data, hora e descrição dos commits**, sendo gerado através do seguinte comando:

```
git log --pretty=format:"%h - %an - %ad - %s" --date=iso > commits.txt
```

O ficheiro deverá ser incluído na pasta do projeto submetido.

Número mínimo de commits

Para demonstrar desenvolvimento progressivo, o projeto deverá conter **no mínimo**:

- Frontend: **15 commits mínimos**
- Backend (PHP): **15 commits mínimos**
- Total mínimo exigido: **30 commits**

No caso de o aluno já ter iniciado o desenvolvimento do projeto antes da publicação do presente documento, serão considerados apenas os commits realizados a **partir da data de publicação**. Adicionalmente, **em função do grau de evolução já atingido**, poderá ser dispensado o cumprimento do mínimo de 15 commits relativos ao frontend, mediante análise

Exemplos de commits aceitáveis (a título exemplificativo)

```
git commit -m "Estrutura inicial das páginas HTML"
git commit -m "Implementação da página de listagem de equipamentos"
git commit -m "Criação do formulário de registo de fornecedor"
git commit -m "Integração de validação JavaScript no formulário"
git commit -m "Implementação da ligação à base de dados"
```



3. Regras de Desenvolvimento

3.1 Regras de Desenvolvimento do Frontend

O desenvolvimento do **frontend** da aplicação web deve respeitar os princípios, tecnologias e boas práticas abordadas nas aulas, garantindo coerência visual, organização estrutural e qualidade da experiência de utilização.

3.1.1. Tecnologias obrigatórias

O frontend deve ser desenvolvido com recurso às seguintes tecnologias:

- HTML5, para a estruturação semântica das páginas;
- CSS3, para a definição da apresentação visual;
- JavaScript, para a implementação de interatividade;
- Bootstrap (versão mais recente), para construção de interfaces responsivas e consistentes.

Poderão ainda ser utilizados recursos complementares, desde que adequadamente integrados, tais como:

- Font Awesome (ícones);
- Chart.js (gráficos);
- DataTables.js (tabelas interativas);
- outras bibliotecas compatíveis com os objetivos do projeto.

3.1.2. Estrutura geral da aplicação

A aplicação deverá estar organizada em duas áreas principais:

- Área pública — acessível sem autenticação;
- Área privada — acessível após autenticação ou simulação de login.

3.1.3. Consistência visual

A aplicação deve apresentar um layout uniforme e coerente em todas as páginas.

Devem manter-se consistentes:

- cores;
- tipografia;
- espaçamentos;
- componentes visuais;
- estrutura de navegação.

O objetivo é garantir uma experiência visual homogénea e profissional.



3.1.4. Ficheiros individuais de CSS e JavaScript

Cada estudante deverá criar ficheiros próprios de estilos e scripts, identificados com o seu número de aluno.

Exemplo:

- assets/css/1940266.css
- assets/js/1940266.js

Estes ficheiros deverão conter exclusivamente:

- estilos utilizados nas páginas desenvolvidas pelo próprio estudante;
- scripts associados às funcionalidades por si implementadas.

3.1.5. Área pública

A área pública deverá incluir pelo menos uma página acessível sem login. Os conteúdos apresentados devem ser realistas e adequados ao contexto da aplicação, refletindo práticas plausíveis em Portugal (nomes, entidades, contactos, horários, serviços, etc.).

A área pública poderá assumir:

- uma única página com navegação por âncoras; ou
- várias páginas simples interligadas.

3.1.6. Gestão de conteúdos da área pública

Os conteúdos da área pública **devem obrigatoriamente** poder ser geridos através da área reservada, evitando a edição direta do HTML.

A aplicação deverá incluir mecanismos que permitam a atualização dinâmica dos conteúdos, garantindo separação entre apresentação e gestão da informação.

Exemplos de funcionalidades a implementar:

- edição de textos;
- atualização de secções;
- alteração de contactos;
- gestão de conteúdos informativos.



3.1.7. Área privada

Após autenticação (real ou simulada), o utilizador deverá ter acesso à área reservada da aplicação. Cada estudante deverá desenvolver uma aplicação funcional completa, correspondente a um sistema de gestão de inventário hospitalar de equipamentos médicos.

Componentes obrigatórios da área privada:

Cada estudante deverá implementar, no mínimo, os seguintes componentes:

▪ Módulo de gestão (CRUD)

O módulo de gestão de dados deverá ser desenvolvido de acordo com o **guião do projeto**, onde se encontram definidos:

- os dados a gerir;
- as entidades envolvidas;
- os requisitos funcionais;
- as operações obrigatórias.

O estudante deverá implementar as funcionalidades de criação, listagem, edição e eliminação de dados, conforme especificado nesse documento.

▪ Dashboard de gestão

Interface com visualização de dados através de:

- gráficos; ou
- indicadores estatísticos.

Os elementos apresentados devem estar relacionados com os dados registados na aplicação, como por exemplo:

- número total de equipamentos;
- equipamentos por categoria;
- equipamentos por localização;
- equipamentos ativos/inativos.

Podem ser utilizadas bibliotecas como:

- Chart.js
- Google Charts



▪ **Backoffice da área pública**

Interface administrativa que permita gerir os conteúdos da área pública da aplicação.

Deverá permitir, nomeadamente:

- alteração de textos;
- atualização de conteúdos informativos;
- gestão de elementos apresentados ao utilizador.

3.1.8. Responsividade

A aplicação deve adaptar-se corretamente a diferentes dispositivos:

- computadores;
- tablets;
- telemóveis.

Deverão ser utilizados os mecanismos de responsividade do Bootstrap, garantindo legibilidade e boa organização em qualquer dimensão de ecrã.

3.1.9. Usabilidade e acessibilidade

A aplicação deverá respeitar princípios básicos de usabilidade e acessibilidade.

Devem ser asseguradas as seguintes boas práticas:

- contraste visual adequado;
- utilização de etiquetas (label) nos formulários;
- identificação clara de campos obrigatórios;
- mensagens de erro claras e específicas;
- destaque de botões e elementos interativos;
- navegação intuitiva;
- indicação visual de estados (ativo, selecionado, erro, etc.).

O objetivo é garantir uma utilização simples, clara e acessível

3.1.10. Utilização de Bootstrap

É obrigatória a utilização de componentes do Bootstrap ao longo da aplicação. Deverá ser incluído pelo menos um componente não explorado nas aulas práticas.

Exemplos:



- Accordion
- Carousel
- Offcanvas
- Toasts
- Progress Bars
- Tooltips

A sua utilização deverá ser funcional e coerente com a aplicação.

3.2 Regras de Desenvolvimento da Base de Dados

A definição da base de dados do projeto deve cumprir os seguintes requisitos:

- Estar na 3ª forma normal (3NF)
- Ser representada em um modelo relacional com notação *Crow's Foot*
- Estar igualmente representada em DBML

O modelo relacional deve ter representadas todas as restrições de integridade necessárias para o projeto e, aquelas que não possam ser representadas no modelo devem ser apresentadas em uma tabela anexa.

O *script* DDL para a criação da base de dados deve ser obtido a partir da representação em DBML, podendo ser posteriormente alterado, se necessário.

A implementação da base de dados para o projeto deve ser feita no servidor utilizado nas aulas práticas do módulo de Bases de Dados, ou seja:

- Servidor: vsgate-s1.dei.isep.ipp.pt:10464
- Credenciais acesso servidor: indicadas nas aulas práticas do módulo
- Base de dados: a que está disponível após acesso servidor

O relatório técnico deve conter um capítulo acerca da base de dados onde deve constar:

- O modelo relacional com as respetivas restrições de integridade
- A restrições de integridade que não podem ser representadas no modelo relacional
- Duas (2) consultas SQL que tenham sido aplicadas no trabalho, uma com junção de tabelas e outra com subconsulta, e respetiva descrição

3.3 Regras de Desenvolvimento do Backend

A aplicação web deverá ser totalmente funcional, segura e bem organizada, garantindo a correta integração entre a área pública, a área privada e a base de dados. Todas as funcionalidades desenvolvidas deverão **respeitar os conteúdos e as boas práticas abordadas nas aulas teóricas e laboratoriais**.

3.3.1. Organização e Estrutura do Código

- O código deve apresentar uma estrutura clara, modular e organizada, com separação lógica das diferentes funcionalidades.
- Devem ser utilizados ficheiros de configuração, funções reutilizáveis e outras estratégias que evitem a duplicação de código.
- O código deverá ser devidamente comentado, identificando as principais secções e explicando as funcionalidades desenvolvidas.
- Devem ser adotadas convenções de nomenclatura consistentes para variáveis, funções e ficheiros.

3.3.2. Interface e Layout da Aplicação

- O layout deve ser responsivo, adaptando-se corretamente a diferentes dispositivos.
- A interface deverá apresentar um design uniforme em toda a aplicação.
- As páginas públicas e privadas devem manter coerência visual e estrutural.

3.3.3. Validação de Dados e Gestão de Formulários

- Os formulários devem ser corretamente estruturados e incluir validação no lado do servidor (PHP).
- Devem ser implementadas técnicas de validação, normalização e sanitização dos dados introduzidos pelo utilizador.
- As mensagens de erro devem ser claras e informativas.
- Sempre que adequado, devem existir páginas intermédias de confirmação (por exemplo, antes da eliminação de um registo).
- Deve ser implementada a funcionalidade de atualização (Update), permitindo alterar corretamente os dados existente

3.3.4. Listagem e Pesquisa de Dados

- Os dados devem ser apresentados em tabelas organizadas, claras e de fácil leitura.
- Devem existir mecanismos de pesquisa e filtragem por diferentes critérios, de acordo com as necessidades da aplicação.

- Sempre que aplicável, poderá ser implementada paginação dos resultados.
- A informação deve encontrar-se organizada e sistematizada, evitando a necessidade de aceder a diferentes páginas para consultar dados relacionados.
- A navegação entre as diferentes funcionalidades deverá ser intuitiva, permitindo ao utilizador localizar rapidamente a informação pretendida.

3.3.5 Gestão de Registos

- Deve ser possível eliminar registos, recorrendo preferencialmente a mecanismos de confirmação prévia ou soft delete.
- A aplicação deverá fornecer mensagens de feedback ao utilizador após as principais operações realizadas.

3.3.6 Dashboards e Gráficos

- Devem ser implementadas métricas relevantes para a aplicação desenvolvida.
- Os dados utilizados nos indicadores devem ser corretamente extraídos da base de dados.
- Devem ser gerados gráficos estatísticos que permitam visualizar tendências ou distribuições.
- Os valores apresentados devem indicar claramente as respetivas unidades de medida.

3.3.7. Segurança e Gestão de Sessões

- O acesso às áreas restritas deverá ser protegido por autenticação.
- As sessões devem ser corretamente geridas através das funcionalidades estudadas nas aulas.
- A funcionalidade de logout deverá destruir adequadamente as variáveis de sessão.
- Nenhuma página protegida deverá ser acessível sem autenticação.

3.3.4. Comunicação com a Base de Dados

- O acesso à base de dados deverá ser realizado de forma segura.
- As passwords devem ser armazenadas utilizando funções adequadas, como `password_hash()`.
- Devem ser aplicadas boas práticas na extração e manipulação dos dados.
- As consultas SQL devem ser construídas de forma segura, evitando vulnerabilidades.

3.3.5. Registo de Eventos (Log Files) e Exportação de Dados

- A aplicação deverá possuir mecanismos de registo de eventos relevantes do sistema, tais como tentativas de autenticação, alterações de dados ou ocorrência de erros.
- Os registos poderão ser armazenados em ficheiros de log ou na base de dados.
- A aplicação deverá ainda permitir a exportação de informação em diferentes formatos, nomeadamente:
 - CSV (Comma Separated Values);

- JSON;
- PDF.

Importante: todas as funcionalidades desenvolvidas deverão estar de acordo com os conteúdos abordados nas aulas teóricas e laboratoriais da unidade curricular.

3.4 Estrutura de Diretórios e Organização do Projeto

A aplicação deverá apresentar uma estrutura organizada, modular e coerente. A estrutura apresentada de seguida corresponde apenas à estrutura mínima recomendada, podendo ser complementada com outras diretorias sempre que tal se justifique.

```
/nomedoprojeto/  
├─ public/  
├─ private/  
├─ assets/  
│   ├─ css/  
│   ├─ js/  
│   └─ img/  
│   . . .  
└─ README.md
```

Regras obrigatórias

- A pasta `public/` deve conter as páginas acessíveis sem autenticação.
- A pasta `private/` deverá conter as páginas protegidas da aplicação, acessíveis apenas após autenticação.
- A pasta `assets/` deve agrupar os recursos estáticos da aplicação, nomeadamente:
 - `css/` — ficheiros de estilos;
 - `js/` — ficheiros JavaScript;
 - `img/` — imagens utilizadas na aplicação
- O ficheiro `README.md` deverá identificar os autores do projeto, a estrutura de diretorias adotada e uma breve descrição da aplicação desenvolvida.

A organização do projeto deverá promover a separação entre conteúdo (HTML/PHP), apresentação (CSS) e comportamento (JavaScript), de acordo com as boas práticas abordadas nas aulas.

Constitui requisito obrigatório que o projeto individual seja executável através do browser, respeitando a seguinte estrutura de endereço:

```
http://127.0.0.1/sibdas/<numeroaluno>/<nome_diretoria_projeto>
```

Importante: O nome da diretoria do projeto deverá ser escrito utilizando apenas letras minúsculas, sem espaços e sem caracteres especiais. Sempre que necessário, as palavras deverão ser separadas por hífen (-)

Por exemplo:

- **Número do aluno:** 1940266
- **Nome da diretoria do projeto:** isep-ginasio

Endereço de execução:

```
http://127.0.0.1/sibdas/1940266/isep-ginasio
```

3.5 Aspetos valorizados no desenvolvimento

Serão valorizados os seguintes aspetos:

- correta aplicação dos conteúdos lecionados;
- organização e clareza da estrutura do projeto;
- coerência visual e funcional;
- utilização adequada de bibliotecas e componentes;
- exploração autónoma de novas soluções;
- criatividade e originalidade.

A utilização de tecnologias ou abordagens não exploradas nas aulas será valorizada, desde que devidamente integrada e justificada no relatório técnico. Será valorizada a coerência global da aplicação, bem como a integração entre as diferentes componentes desenvolvidas.

3.6. Regras de Submissão

A submissão do projeto individual deverá ser efetuada exclusivamente através da plataforma Moodle.

Cada estudante deverá submeter um único ficheiro comprimido (.ZIP), respeitando as seguintes regras:

Regras gerais:

- A submissão deverá ser realizada exclusivamente através da plataforma Moodle.
- O nome do ficheiro ZIP deverá corresponder ao nome da diretoria do projeto.
- Exemplo:
 - **isep-ginasio.zip**
 - O ficheiro ZIP deverá conter toda a estrutura de diretórios e ficheiros necessários ao correto funcionamento da aplicação.
 - O projeto submetido deverá ser executável sem necessidade de alterações estruturais por parte do docente.
 - O ficheiro ZIP deverá incluir uma cópia da base de dados em formato .sql, obtida através da funcionalidade de exportação do sistema de gestão de bases de dados utilizado.
 - A importação do ficheiro .sql deverá permitir recriar integralmente a estrutura e os dados necessários ao funcionamento da aplicação.

Exemplo do conteúdo mínimo do ficheiro ZIP:

- Projeto completo;
- Base de dados (ficheiros DBML e *script* DDL);
- Ficheiro README.txt;
- Relatório técnico individual em formato PDF.

Conteúdo obrigatório do ficheiro README.txt

O ficheiro README.txt deverá incluir, obrigatoriamente:

- Nome do projeto;
- Nome do estudante;
- Número do estudante;
- Instruções para instalação e execução da aplicação;
- Instruções para realização dos principais testes da aplicação;
- Credenciais de acesso correspondentes a todos os perfis existentes na aplicação (por exemplo: administrador, técnico, rececionista, cliente, etc.);
- Qualquer informação adicional considerada relevante para a avaliação do projeto.

Importante:

A ausência do ficheiro README.txt, a inexistência de credenciais válidas ou a impossibilidade de executar a aplicação poderão comprometer a avaliação integral do projeto.

Grelha Geral de Avaliação do Projeto Individual

Critério	Peso (%)	Descrição
1. Funcionamento geral da aplicação	20%	A aplicação encontra-se funcional e cumpre os requisitos definidos no enunciado. As páginas públicas e privadas apresentam as funcionalidades previstas, permitindo executar corretamente as principais operações do sistema e garantindo a integração entre os diferentes módulos da aplicação.
2. Base de dados	10%	A base de dados apresenta uma estrutura adequada, com tabelas coerentes, relações corretamente definidas e dados suficientes para testar a aplicação. O ficheiro .sql permite recriar integralmente a estrutura e os dados necessários.
3. Operações CRUD e formulários	10%	A aplicação permite inserir, listar, editar e apagar registos. Os formulários encontram-se devidamente estruturados e processam corretamente os dados introduzidos.
4. Validação, segurança e sessões	15%	Existem mecanismos de validação no lado do servidor, autenticação, gestão de sessões, encerramento correto da sessão (logout) e proteção contra acessos diretos a páginas reservadas.
5. Estrutura e organização do código	15%	O código encontra-se organizado de forma modular, com separação clara entre ficheiros, diretorias e responsabilidades. Existe reutilização de código, modularização e boa legibilidade.
6. Interface, navegação e usabilidade	10%	A aplicação apresenta uma interface coerente, responsiva e de fácil utilização. A navegação é intuitiva e a informação encontra-se organizada de forma clara, evitando redundâncias e dispersão desnecessária dos dados.
7. README, submissão e relatório individual	10%	O ficheiro README.txt contém instruções claras de instalação, execução, testes e credenciais de acesso. O relatório individual descreve o trabalho realizado, as opções técnicas adotadas e as principais dificuldades encontradas.
8. Funcionalidades adicionais e valor acrescentado	10%	Implementação de funcionalidades adicionais devidamente integradas na aplicação, demonstrando iniciativa, criatividade e preocupação com a melhoria da experiência de utilização, da organização da informação e da gestão dos dados. Sempre que adequado, poderão ser implementadas funcionalidades como exportação de dados, métricas, dashboards, gráficos, estatísticas ou outras soluções que acrescentem valor ao projeto.

Total: 100%



Nota importante relativa à avaliação individual

- A descrição apresentada em cada critério da grelha de avaliação é meramente indicativa e exemplificativa, não sendo exaustiva.
- Em cada critério poderão ser considerados diversos parâmetros de avaliação relacionados com os conteúdos lecionados nas aulas teóricas e laboratoriais, bem como as boas práticas de desenvolvimento abordadas ao longo da unidade curricular.
- A classificação atribuída em cada um dos critérios pressupõe que o estudante demonstre conhecimento e domínio das soluções implementadas.
- Durante a apresentação e defesa do projeto, serão colocadas questões técnicas relacionadas com o código desenvolvido, a estrutura da base de dados, as funcionalidades implementadas e as opções técnicas adotadas.
- A avaliação não incide apenas sobre o resultado final obtido, mas também sobre a capacidade do estudante em compreender, justificar e explicar as soluções apresentadas.
- Assim, a incapacidade de explicar adequadamente componentes específicas do projeto poderá conduzir à desvalorização parcial ou total dos critérios correspondentes. Por exemplo:
 - dificuldades na explicação da estrutura da base de dados poderão refletir-se no critério Base de Dados;
 - dificuldades na justificação das operações de inserção, listagem, alteração e eliminação de dados poderão refletir-se no critério Operações CRUD e Formulários;
 - dificuldades na explicação dos mecanismos de autenticação, sessões e segurança poderão refletir-se no critério Validação, Segurança e Sessões;
 - dificuldades na justificação da estrutura do projeto, da organização das diretorias e da modularização do código poderão refletir-se no critério Estrutura e Organização do Código.
- Em casos em que o estudante demonstre um conhecimento muito reduzido sobre as soluções implementadas, a desvalorização poderá abranger vários critérios da grelha de avaliação